



APRENDIZAJE DE MÁQUINA

ÁRBOLES DE DECISIÓN

Especialización en Inteligencia Artificial
FIUBA

Dr. Ing. Facundo Adrián Lucianna
Esp. Lic. María Carina Roldán

En la clase anterior vimos ...

Máquinas de vectores de soporte. Maximal Margin Classifier. Clasificador de Vector de soportes. Máquinas de vectores de soporte en regresión.

MAXIMAL MARGIN CLASSIFIER

En general si nuestra data es linealmente separable, puede existir un número infinito de hiperplanos que van a poder separa las clases.

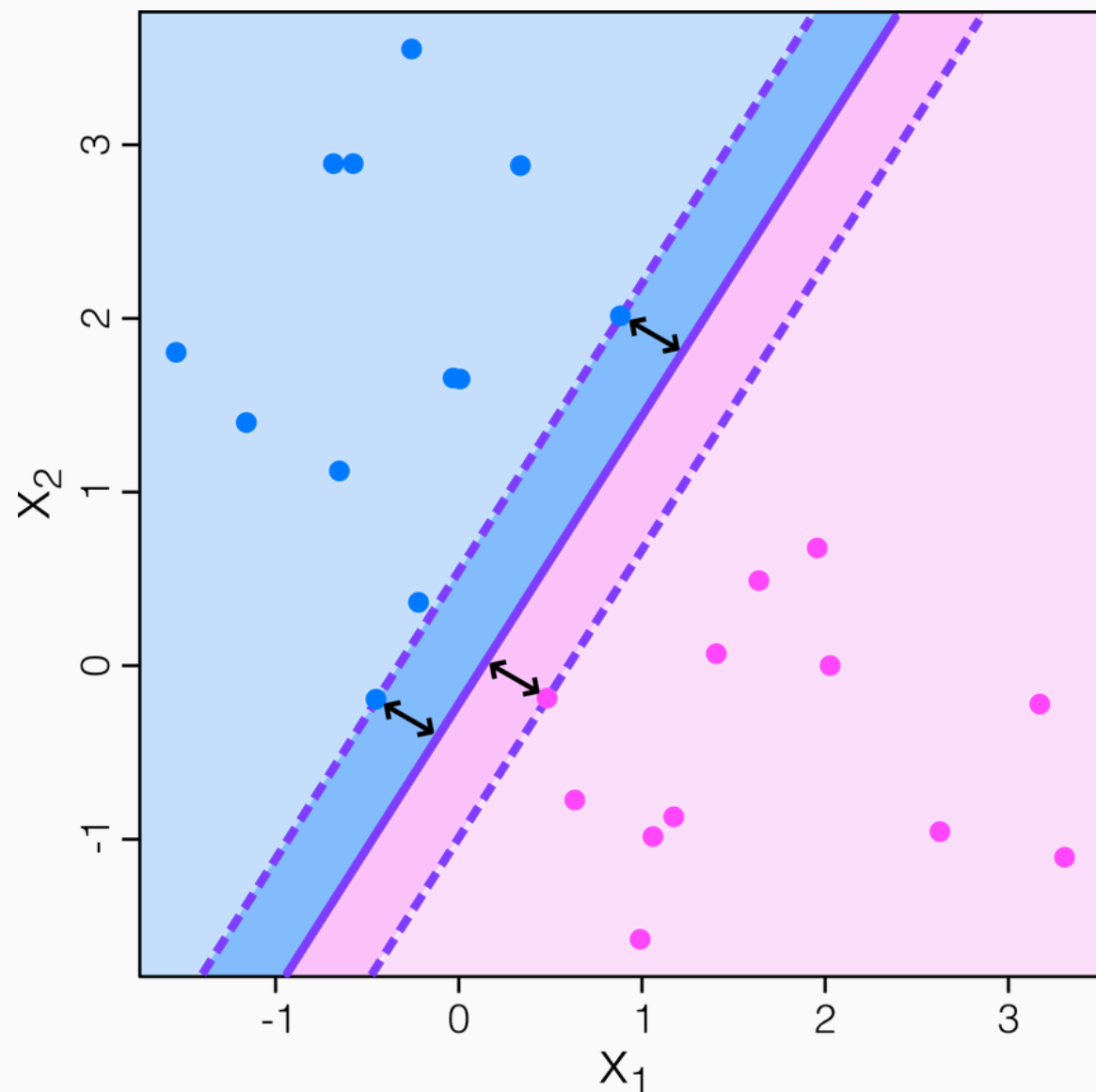
Por lo que necesitamos algún criterio de selección.

El caso que aquí estamos viendo busca el hiperplano que más lejos se encuentra de los datos de entrenamiento.

Es decir, computamos la distancia mínima de cada observación de entrenamiento y obtenemos la distancia más chica de las distancias, que llamamos margen.

El objetivo es buscar el hiperplano con mayor margen y, el algoritmo que hace esto es el **Maximal Margin Classifier**.

Podemos pensar que el clasificador busca, entre las rectas que pueden pasar entre las clases, la de máximo **grosor**.



ÁRBOL DE CLASIFICACIÓN

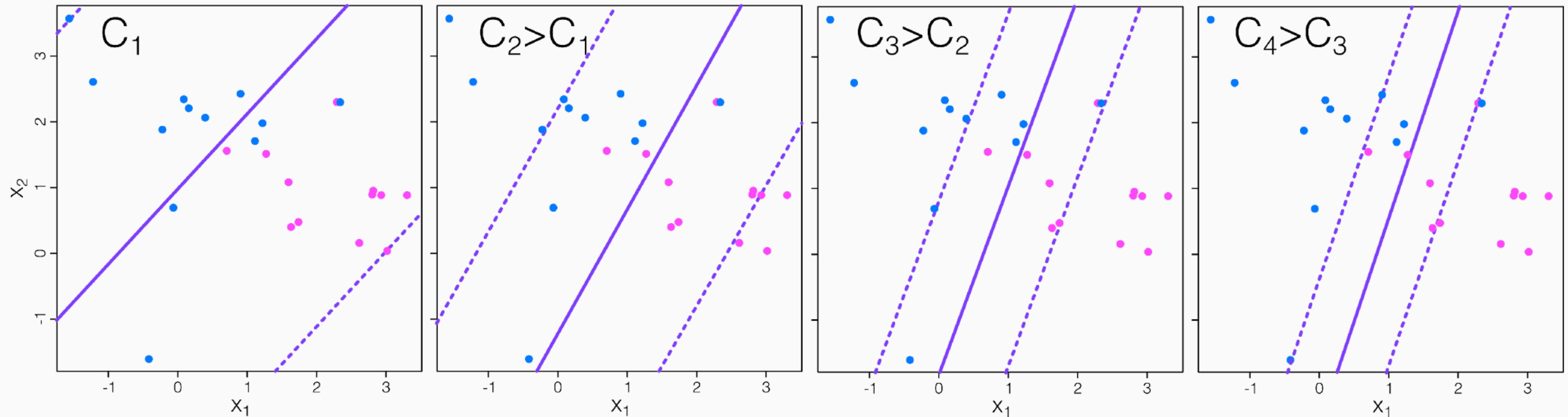
Por lo que vimos, si queremos seguir usando un hiperplano, debemos relajar las exigencias:

- Mayor robustez a observaciones individuales.
- **Mejor clasificación de la mayoría** (no todas) de las observaciones de entrenamiento.

Es decir, podría valer la pena **clasificar erróneamente algunas** observaciones de entrenamiento para poder clasificar mejor las observaciones restantes.

El modelo clasificador de vectores de soporte es el modelo apropiado para este caso. Con este modelo una observación puede estar no solo en el lado equivocado del **margen**, sino también en el **lado equivocado del hiperplano**.

SOFT-MARGIN SVM



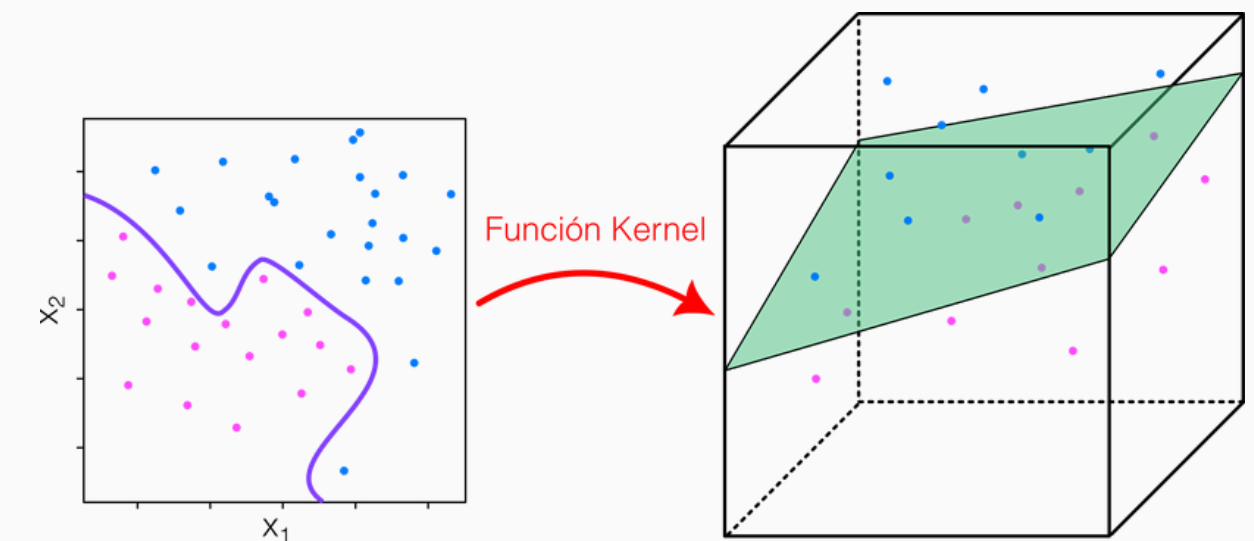
- Hay un pequeño número de observaciones en el margen o dentro de él, que son los que determinan el hiperplano, esto se llaman vectores de soporte.
- El valor de C nos da un trade-off (compensación) entre sesgo y varianza.

SUPPORT VECTOR MACHINE

El modelo llamado **Support Vector Machine (SVM)** o máquina de vector de soportes extiende el Clasificador de Vector de Soportes permitiendo ampliar el espacio de atributos, usando funciones **kernels**.

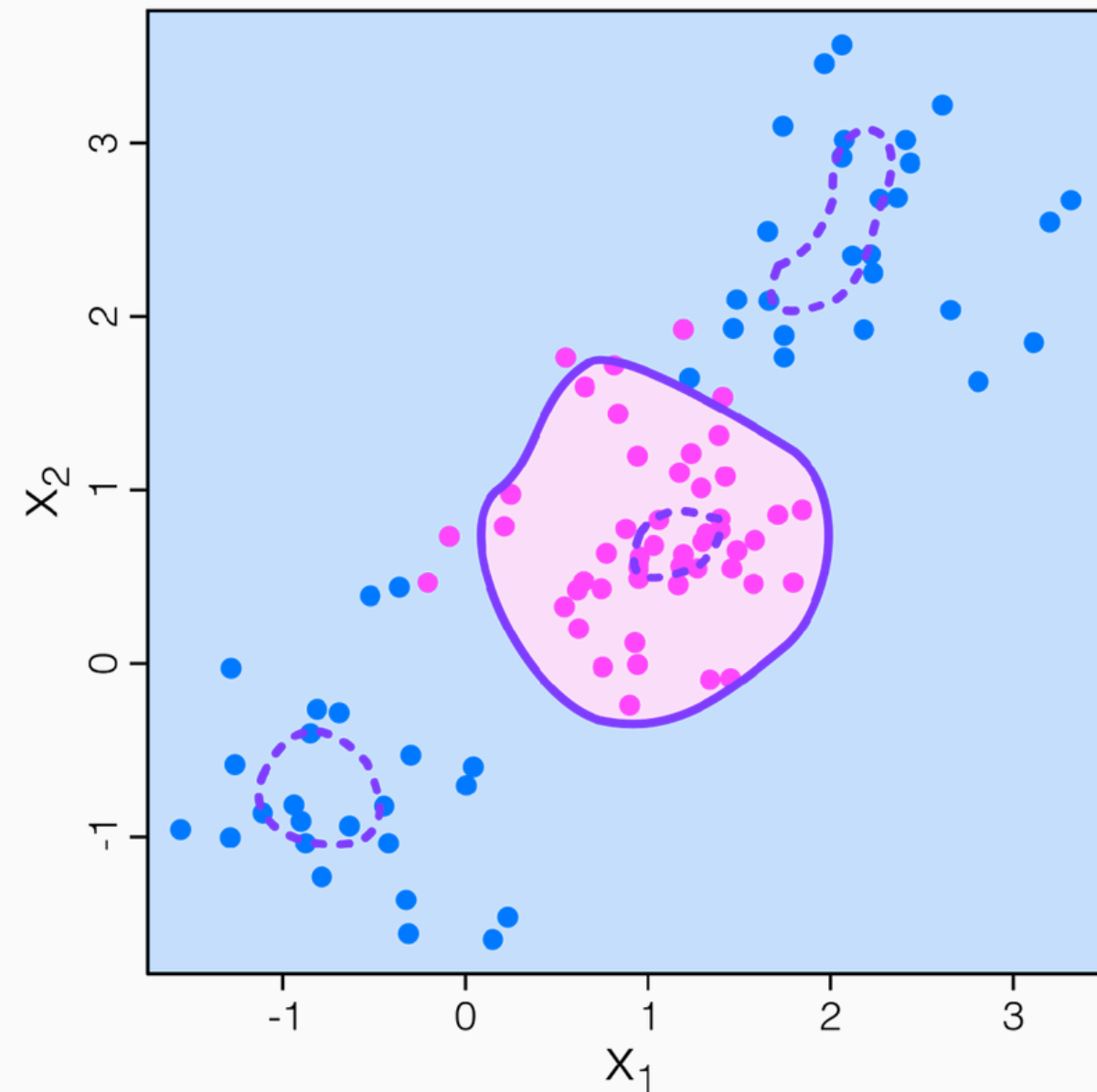
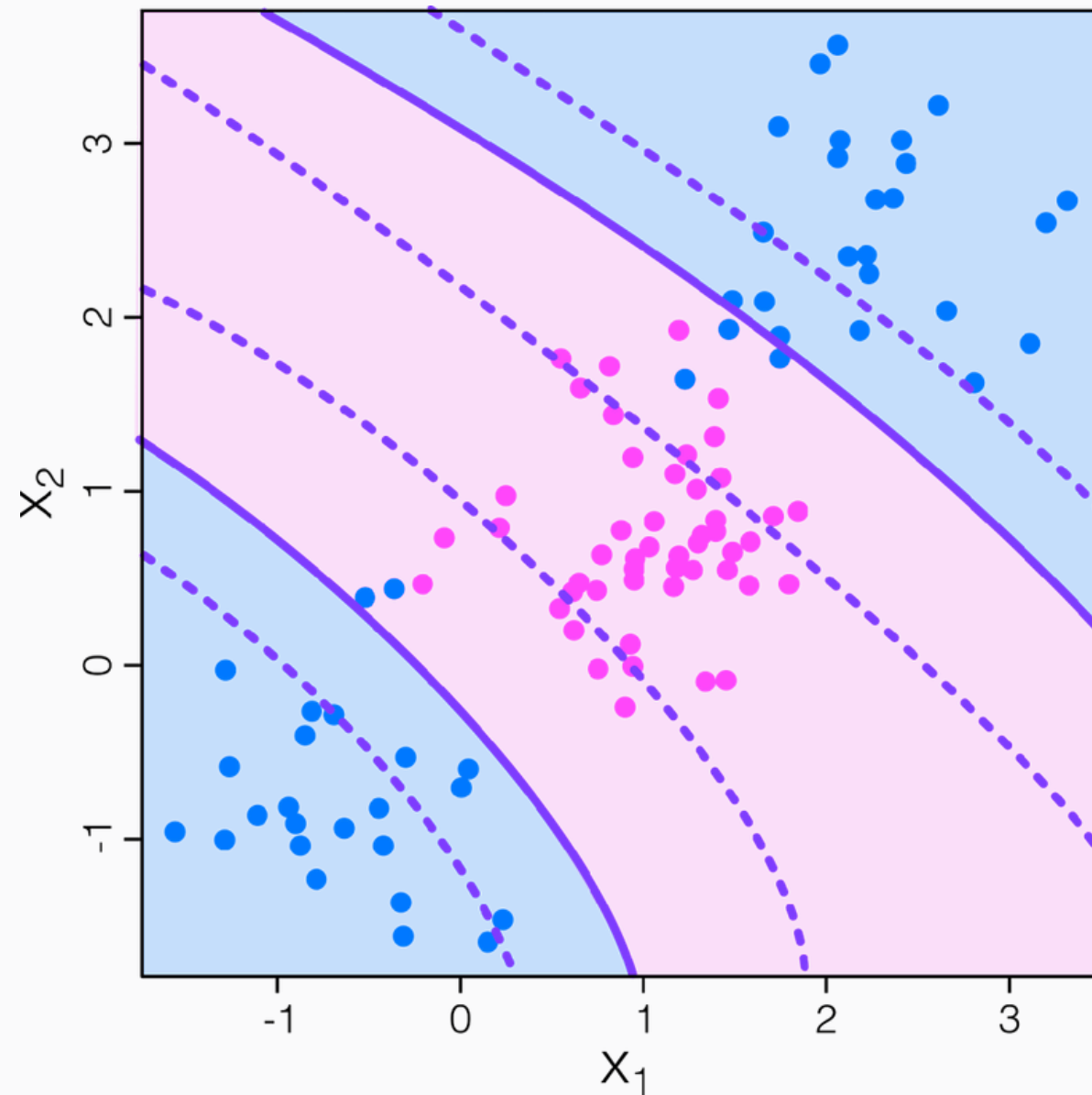
En general, podemos escribir la función de decisión de la máquina de vector de soportes como:

$$f(x) = \beta_0 + \sum_{i \in H} \alpha_i K(X_i, X)$$



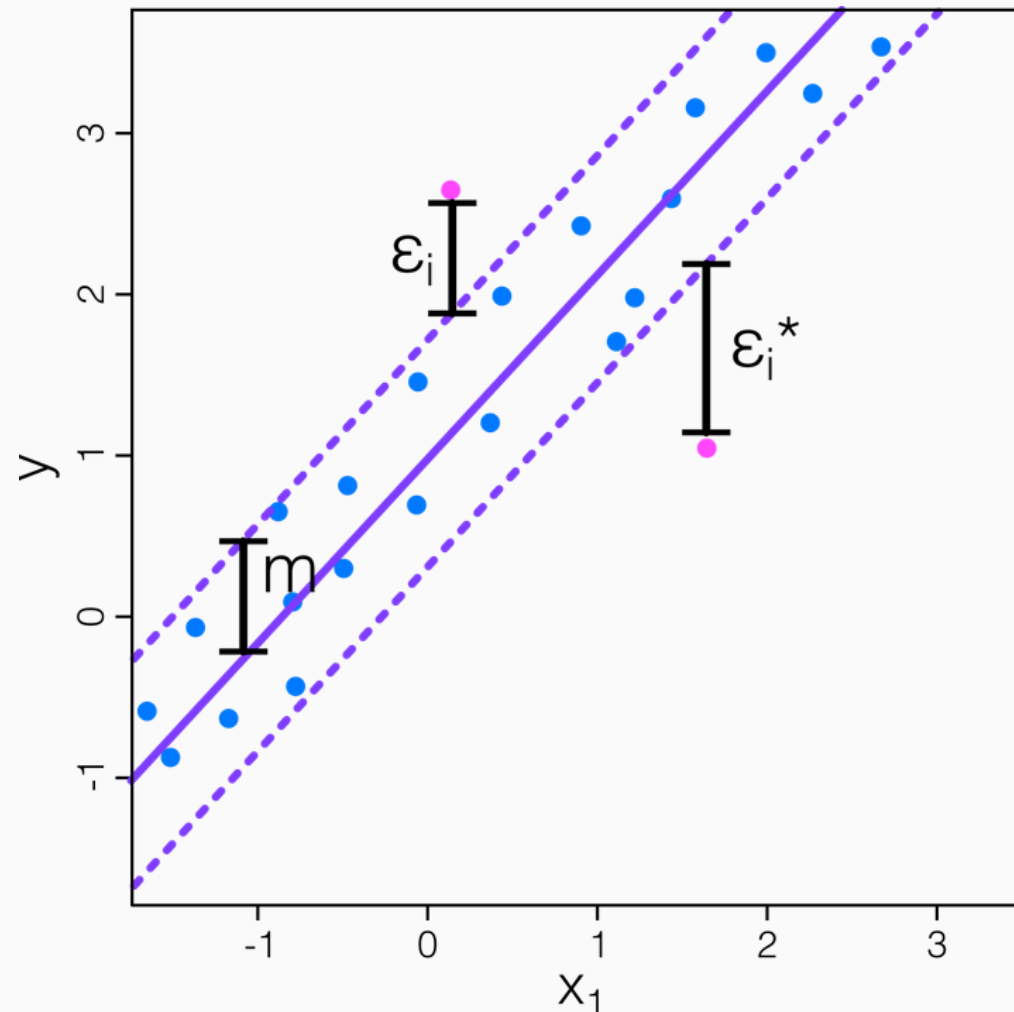
La frontera de decisión no es necesariamente lineal sino que su forma queda determinada por la **función de kernel elegida**.

SOFT-MARGIN SVM



SUPPORT VECTOR MACHINE EN REGRESIÓN

En regresión, lo que se optimiza ahora es el hiperplano que mejor que logra meter a todos los puntos de entrenamiento dentro del margen, minimizando el valor del margen.



$$\sum_{i=1}^n (\varepsilon_i + \varepsilon_i^*) \leq \frac{1}{C} \quad \varepsilon_i, \varepsilon_i^* \geq 0 \quad \forall i = 1, \dots, n$$

y la restricción es:

$$y_i - (b_0 + \langle \vec{b}, X \rangle) \leq m + \varepsilon_i$$

$$(b_0 + \langle \vec{b}, X \rangle) - y_i \leq m + \varepsilon_i^*$$

$$\forall i = 1, \dots, n$$

ÁRBOLES DE DECISIÓN

ÁRBOLES DE DECISIÓN

Los árboles de clasificación y regresión, conocidos como **CART** (*Classification and Regression Trees*), son una poderosa técnica de aprendizaje automático que se utiliza ampliamente para resolver problemas tanto de **clasificación** como de **regresión**.

Los CART representan el proceso de decisión mediante una estructura en forma de árbol. A partir de una secuencia de reglas simples, permiten realizar predicciones de manera intuitiva y fácilmente interpretable.



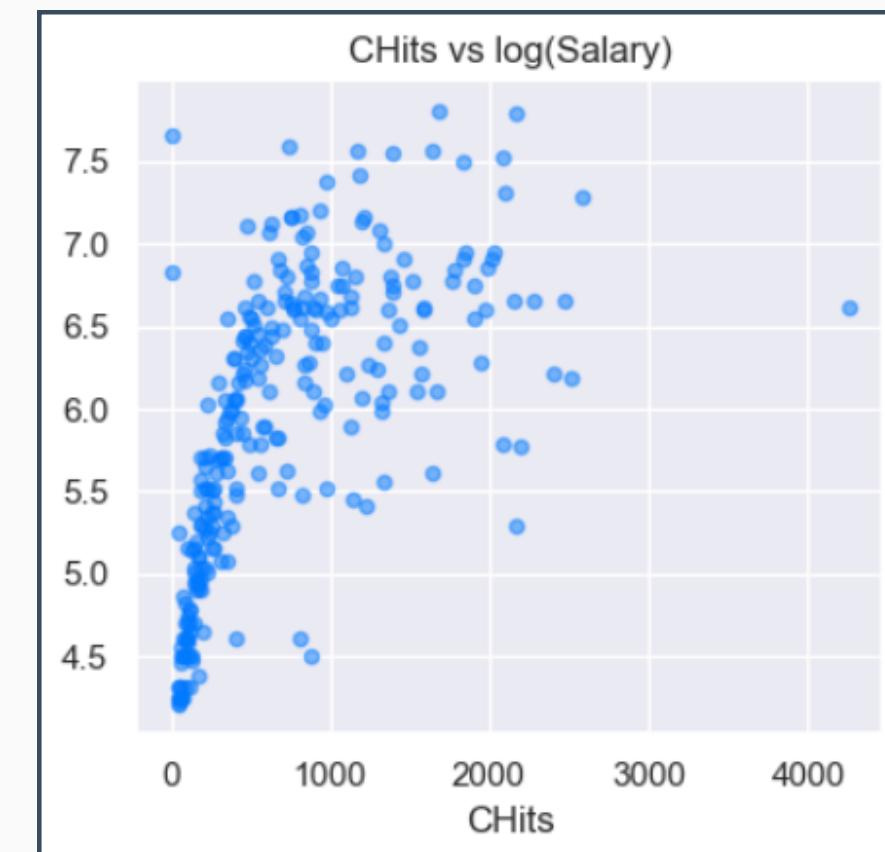
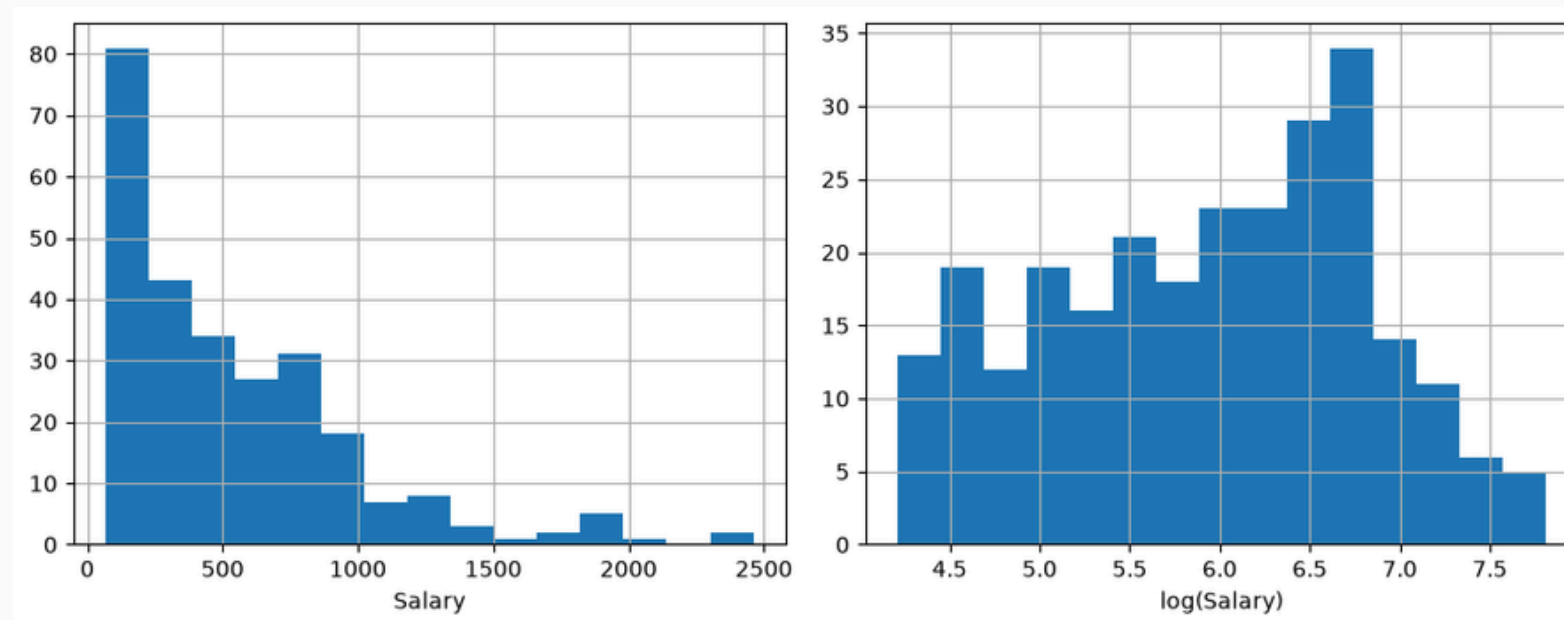
ÁRBOLES DE
CLASIFICACIÓN

ÁRBOLES DE
REGRESIÓN

ÁRBOLES DE REGRESIÓN

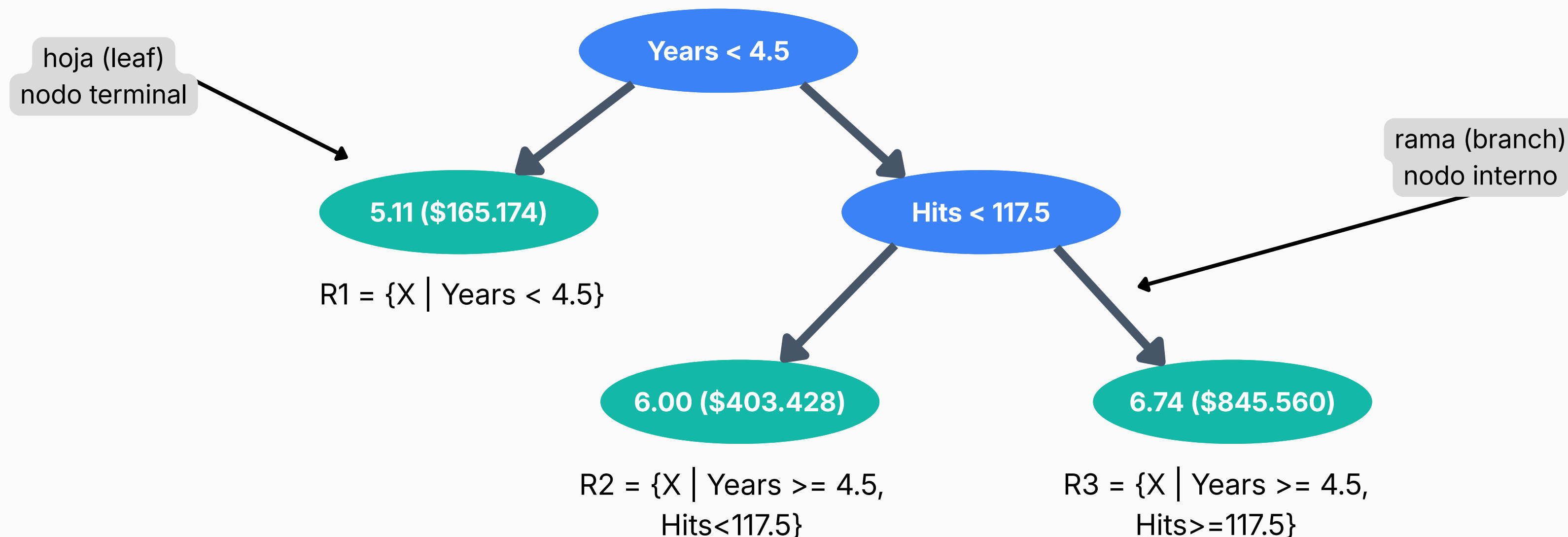
Empecemos con un ejemplo sencillo, usando el dataset Hitters (Dato de salarios de jugadores de Béisbol de 1987 y estadísticas deportivas de 1986).

- Vamos a predecir el logaritmo del salario.
- Years corresponde a los años jugando en las ligas.
- Hits es el número de hits que realizó el año pasado.



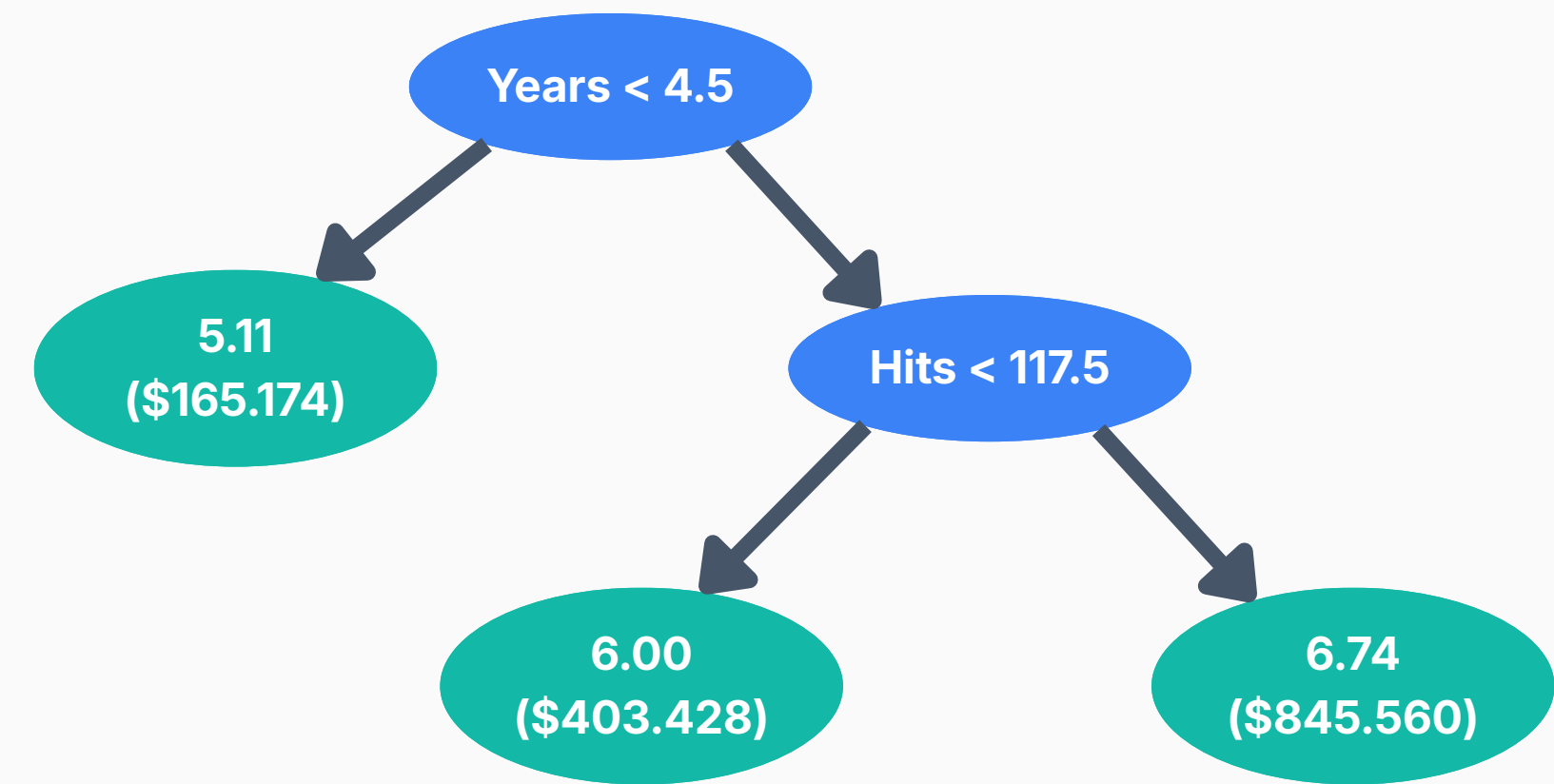
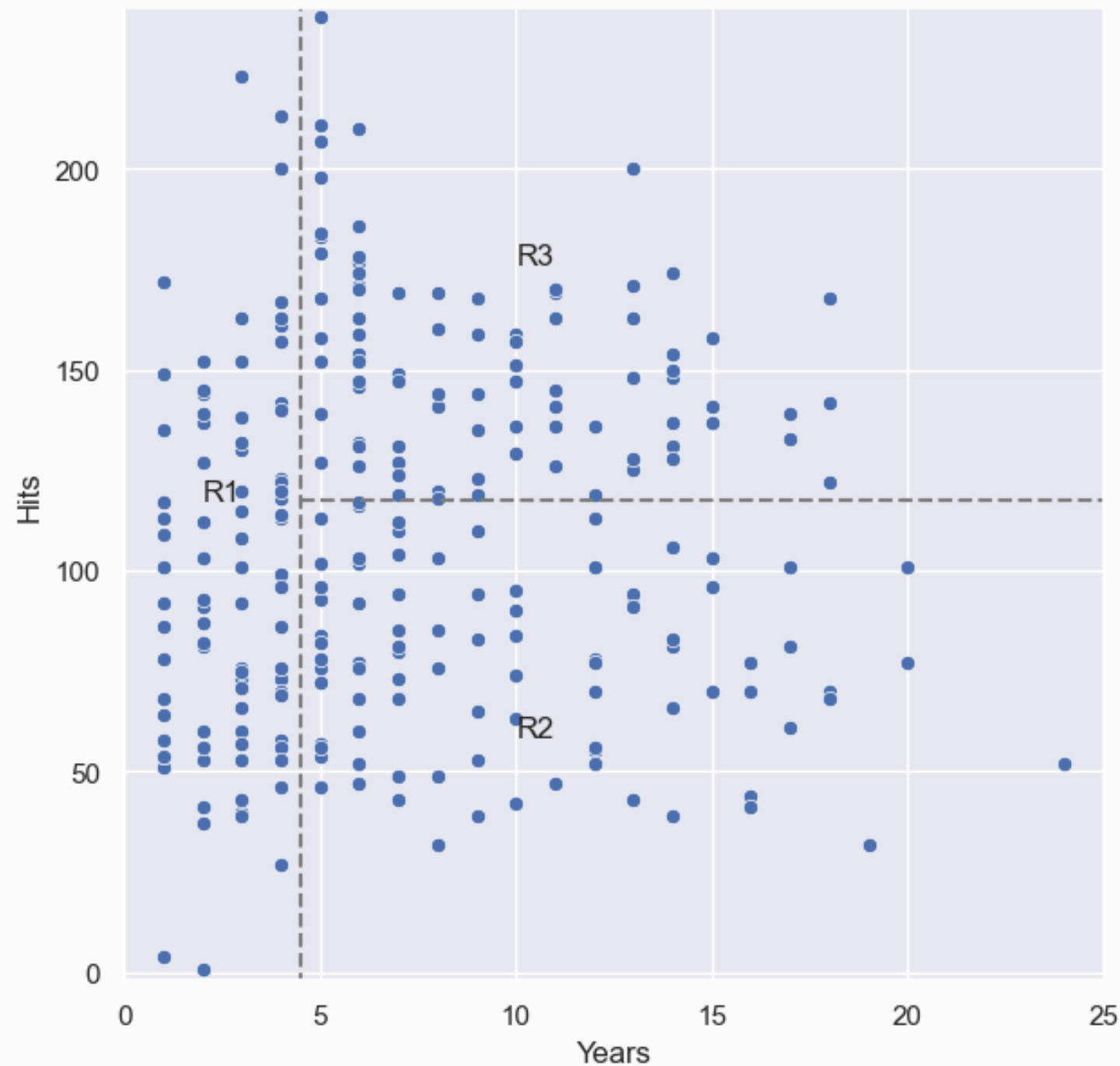
ÁRBOLES DE REGRESIÓN

El **árbol de regresión** para nuestro ejemplo es una sobre simplificación del verdadero valor de regresión entre Salary, Years y Hits. Sin embargo, tiene sus ventajas porque es más fácil entender y tienen mejor representación gráfica.



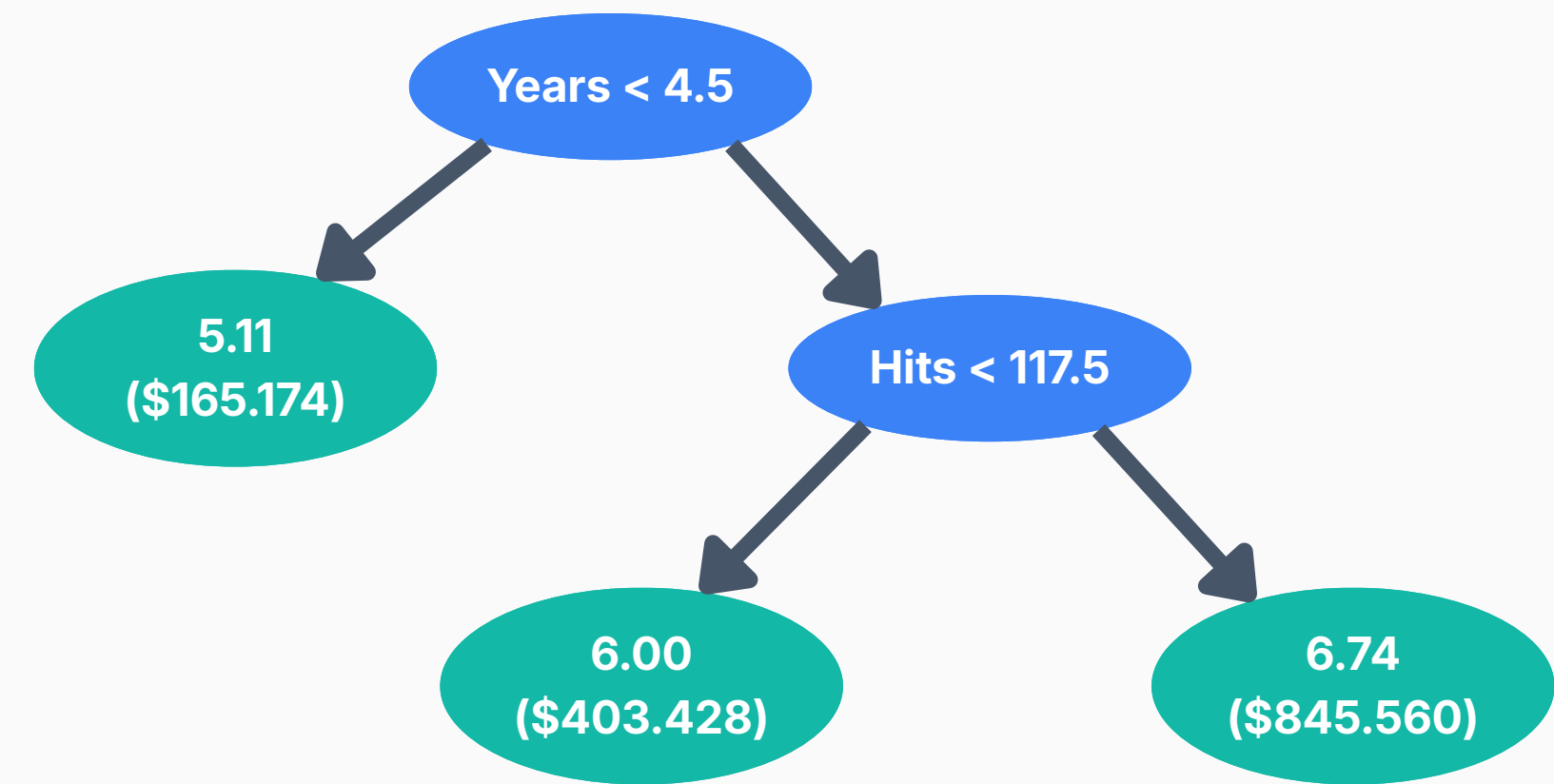
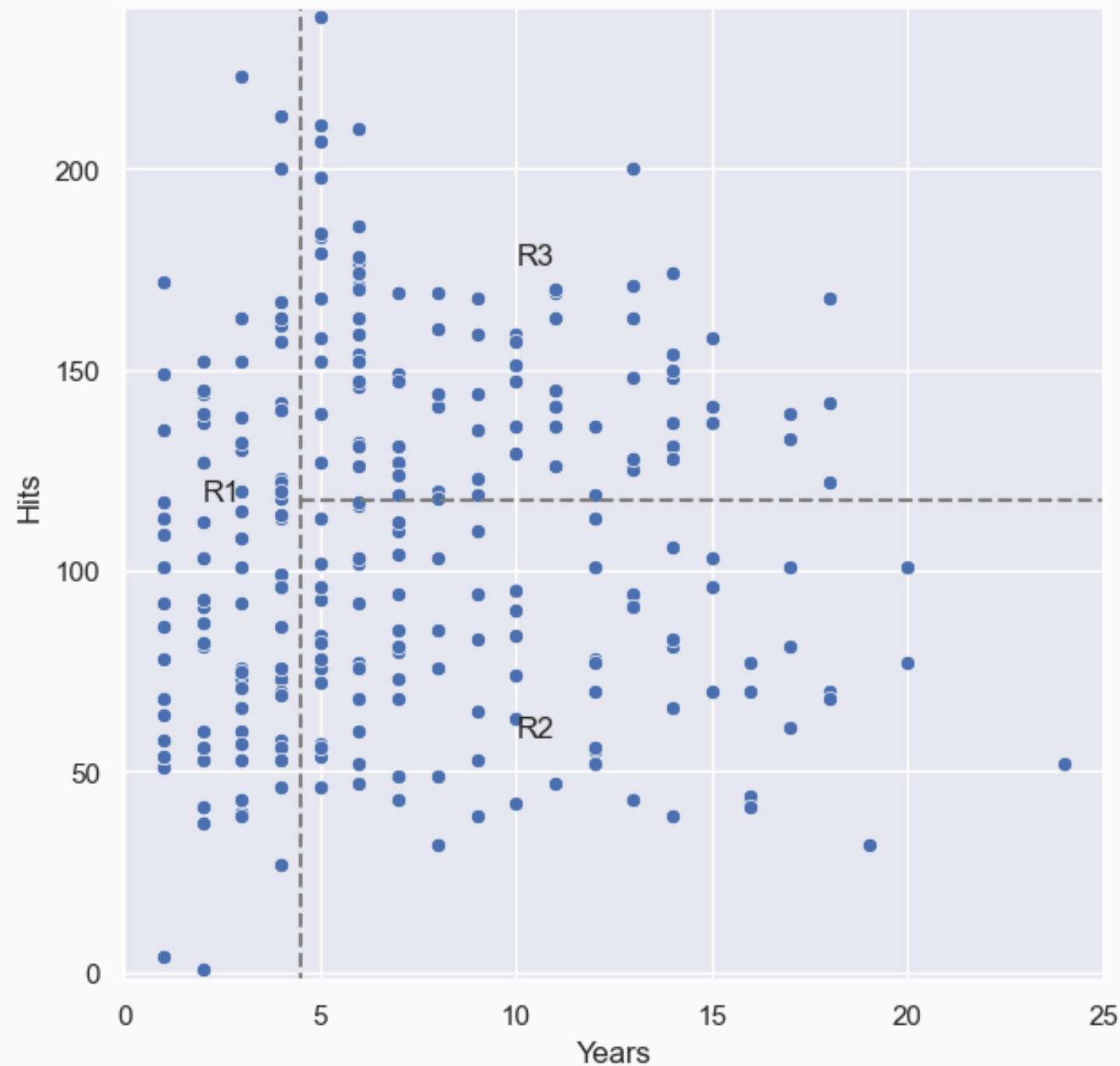
ÁRBOLES DE REGRESIÓN

Durante el **entrenamiento**, el algoritmo construye un árbol dividiendo el espacio de observaciones en regiones cada vez más homogéneas.



ÁRBOLES DE REGRESIÓN

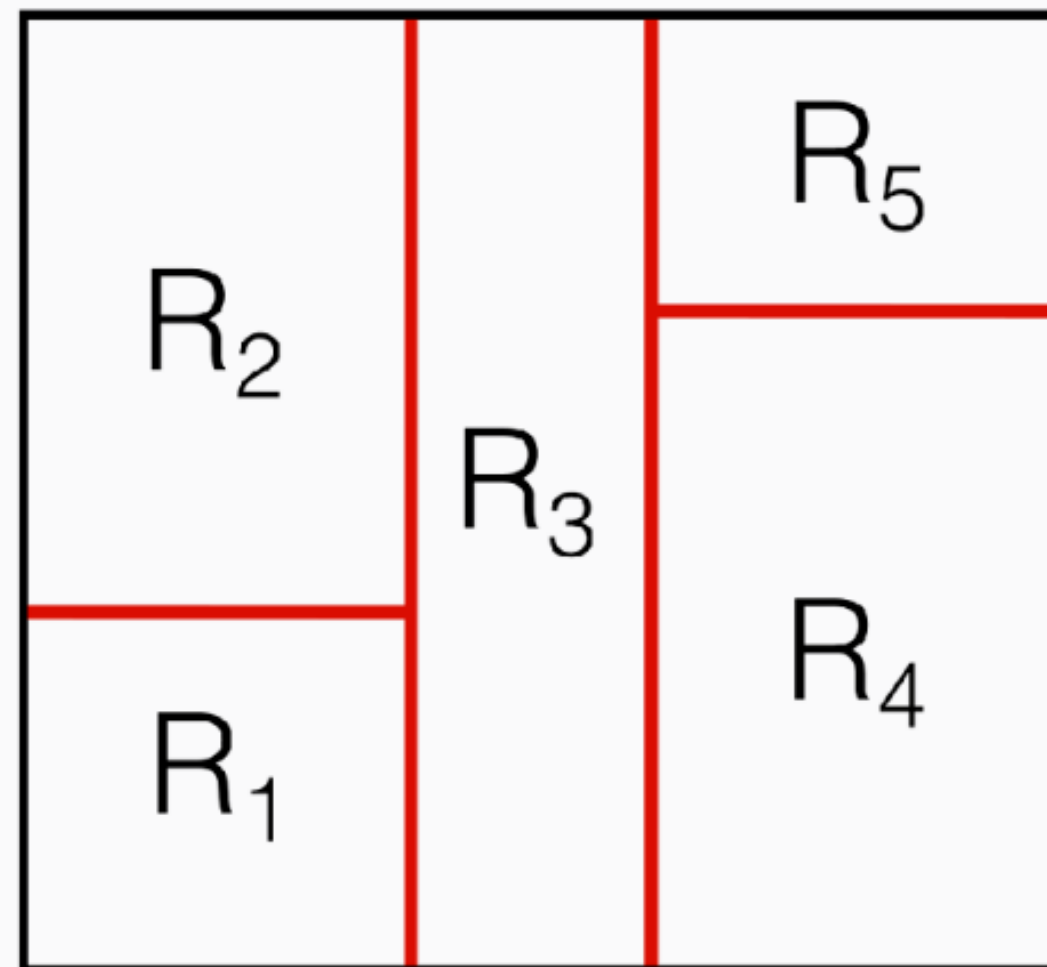
Para **predecir** una nueva observación, se sigue el recorrido del árbol hasta una hoja, donde recibe como predicción el valor representativo de esa región. En regresión suele ser la media de las observaciones de entrenamiento.



ÁRBOLES DE REGRESIÓN

¿Cómo se construye el árbol?

- Dividimos el espacio de observaciones (set de los valores posibles) $X_1, X_2, \dots, X_p$ en J regiones disjuntas $R_1, R_2, \dots, R_J$.



ÁRBOLES DE REGRESIÓN

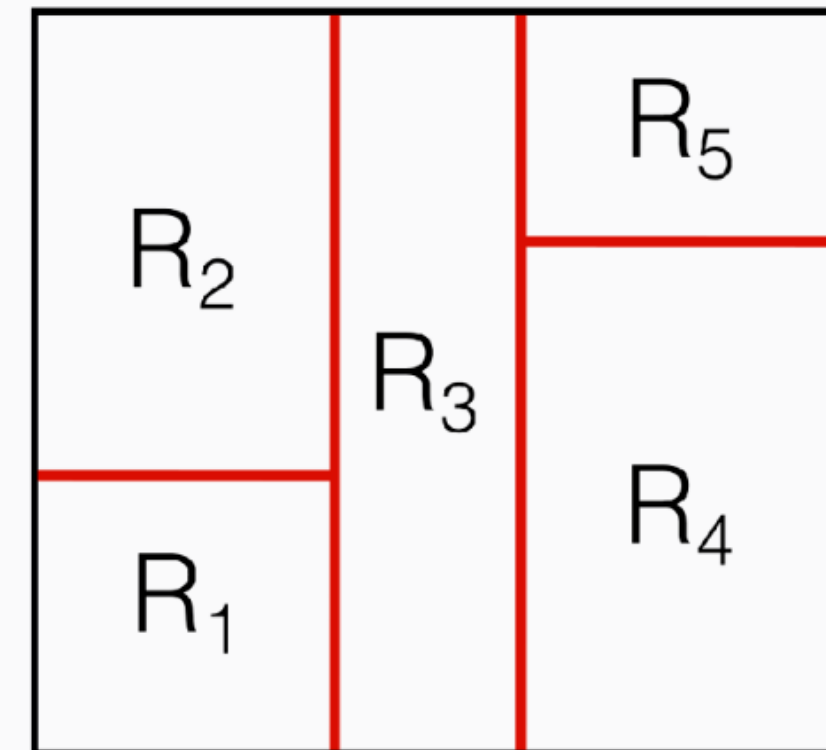
¿Cómo se divide el espacio de observaciones?

En teoría, lo podríamos dividir en cualquier tipo de regiones pero, para simplificar el modelo, se eligen espacios rectangulares.

El objetivo es encontrar cajas $R_1, R_2, \dots, R_J$ que minimicen la suma al cuadrado de los residuos dada por:

$$RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

media de y en
la región R_j



ÁRBOLES DE REGRESIÓN

¿Cómo se divide el espacio de observaciones?

Es imposible buscar todas las combinaciones posibles de valores para minimizar RSS . Tenemos que usar algún algoritmo de optimización.

Vamos a usar un algoritmo top-down greedy llamado **Recursive binary splitting**:

- top-down: empezamos desde el tronco del árbol y vamos bajando.
- greedy: En cada paso, se busca la mejor bifurcación en ese paso particular.

ÁRBOLES DE REGRESIÓN

Algoritmo Recursive binary splitting

- Elegimos un X_j y un punto de corte s de tal forma que bifurcan el espacio de features en dos regiones $\{X | X_j < s\}$ y $\{X | X_j \geq s\}$ de manera que la reducción de RSS sea máxima.

Es decir, para cada valor de j y cada valor de s :

$$R_1(j, s) = \{X | X_j < s\} \quad R_2(j, s) = \{X | X_j \geq s\}$$

- Buscamos el valor de j y s que minimice esta ecuación:

$$\sum_{i: X_i \in R_1(j, s)} (y_i - y_{R_1})^2 + \sum_{i: X_i \in R_2(j, s)} (y_i - y_{R_2})^2$$

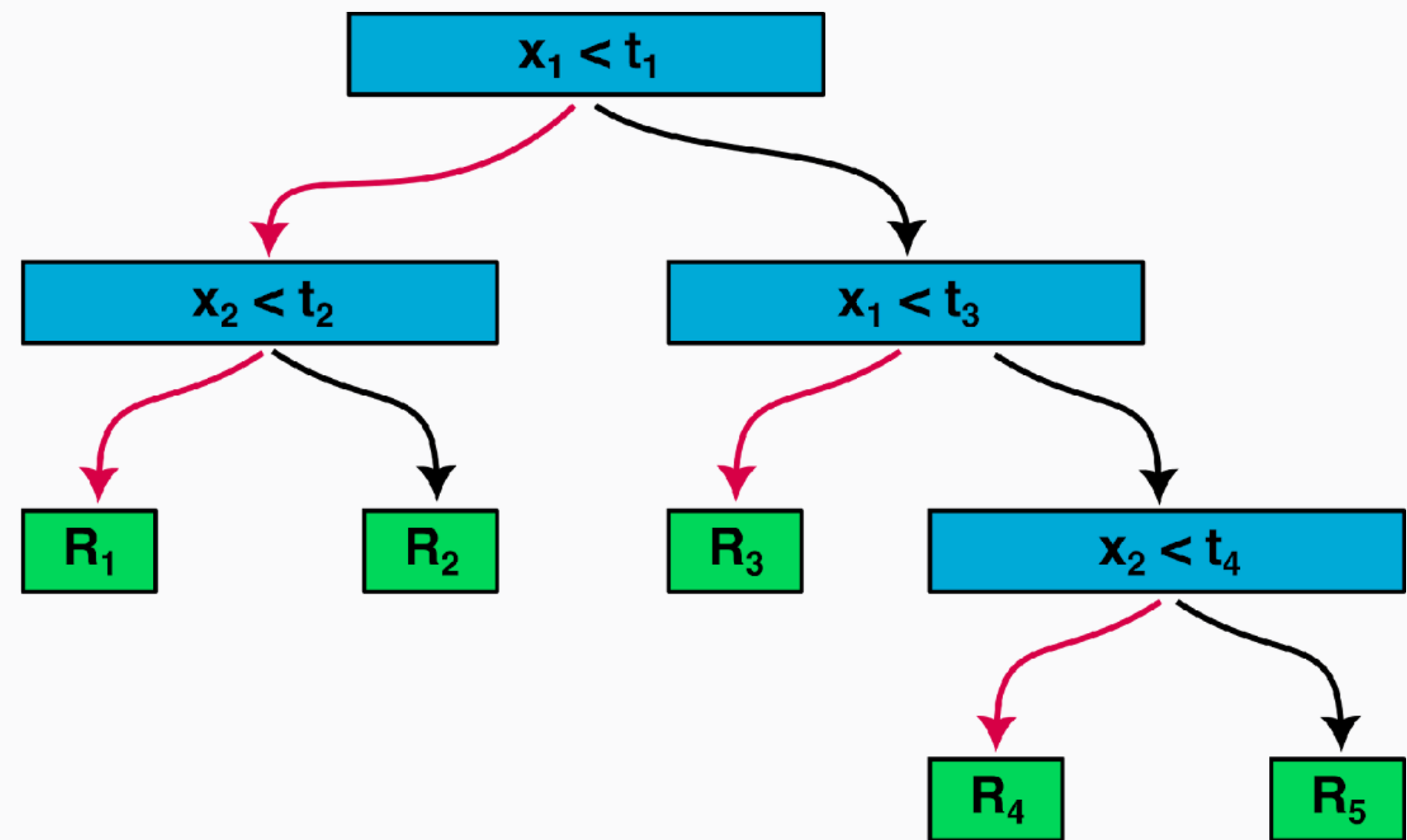
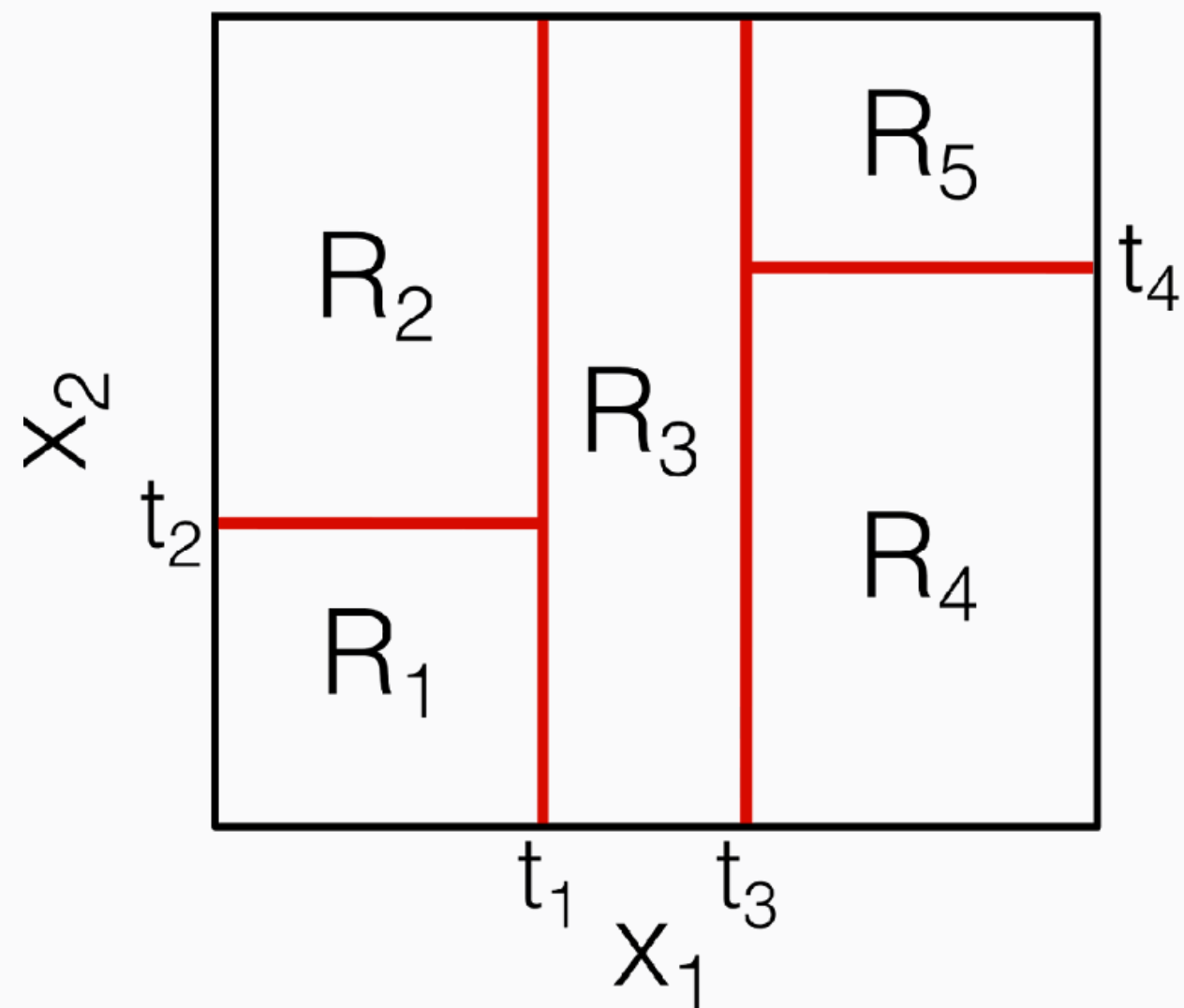
ÁRBOLES DE REGRESIÓN

Algoritmo Recursive binary splitting

- El proceso continúa de manera recursiva: cada región generada se vuelve a dividir aplicando el mismo criterio de partición.
- Seguimos repitiendo los pasos hasta que llegamos a un **criterio de corte**.

ÁRBOLES DE REGRESIÓN

Algoritmo Recursive binary splitting



ÁRBOLES DE REGRESIÓN

Podando los árboles

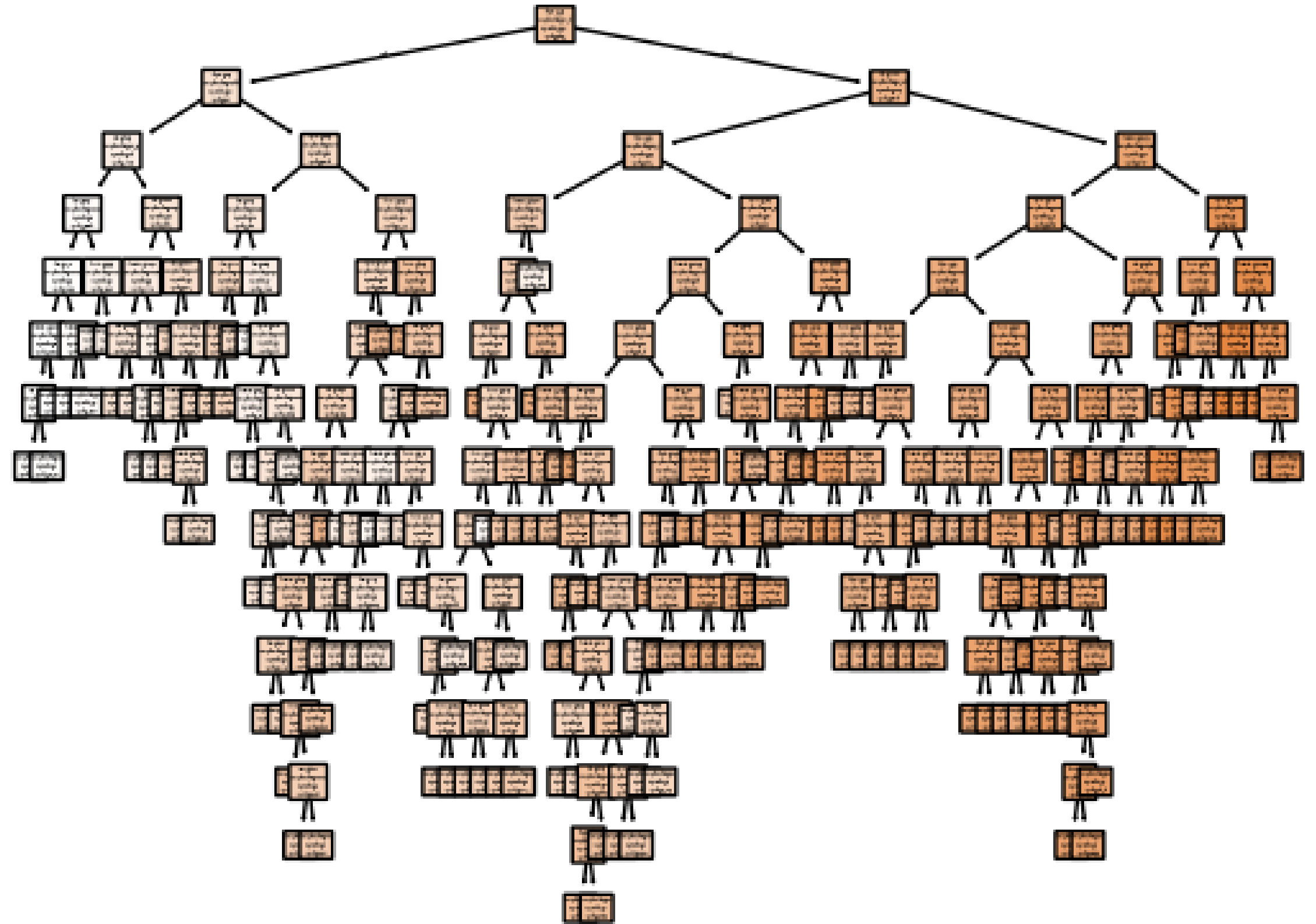
El proceso que se describió puede producir buenas predicciones del set de entrenamiento, pero muy fácilmente puede generar **overfitting**, haciendo que se desempeñe muy mal en el set de validación.

Esto se debe a que el árbol es muy complejo. Un árbol más pequeño con menor regiones puede llevar a menos **varianza** y mejor interpretación a expensa de un poco de **sesgo**.

ÁRBOLES DE REGRESIÓN

Podando los árboles

El caso más extremo es un árbol con una hoja por cada punto del set de entrenamiento.



ÁRBOLES DE REGRESIÓN

Podando los árboles

La estrategia más obvia es construir el árbol solo mientras la disminución en el **RSS** sea mayor a un valor umbral (relativamente alto), llamado **early stopping**.

Esta estrategia dará como resultado árboles más pequeños, pero es demasiado cortoplacista. Una división aparentemente sin valor en las primeras etapas del árbol podría ser seguida por una división muy buena, es decir, una división que conduzca a una gran reducción del RSS más adelante.

ÁRBOLES DE REGRESIÓN

Podando los árboles

Una mejor estrategia es construir un árbol enorme T_0 y luego aplicar un proceso de poda *de abajo hacia arriba* para obtener un subárbol.

Intuitivamente, el objetivo es elegir el subárbol que minimice el error de validación.

Pero, una vez más, **evaluar todos los subárboles posibles es un problema combinatorio demasiado complejo de calcular.**

ÁRBOLES DE REGRESIÓN

Podando los árboles

Vamos a introducir un valor de penalización α en nuestra fórmula de RSS.
Dado un valor de α , existe un subárbol T perteneciente a T_0 tal que:

$$RSS = \sum_{m=1}^L \sum_{i \in R_m} (y_i - y_{R_m})^2 + \alpha L$$

$$\alpha \geq 0$$

L número de hojas del
subárbol

RSS es mínimo.

ÁRBOLES DE REGRESIÓN

Podando los árboles

α presenta un trade-off entre complejidad del árbol y su capacidad de ajustar a los datos (regularización).

- Si $\alpha = 0$, el árbol elegido es T_0 .
- Cuanto más grande es α , el precio a pagar por el tamaño del árbol es mayor.

Algo interesante es que:

- Al aumentar α , las ramas se podan de forma anidada y predecible.
- Esto permite obtener fácilmente la secuencia completa de subárboles en función de α
- El α final se elige mediante búsqueda de hiperparámetros (ej: Validación Cruzada).

ÁRBOLES DE REGRESIÓN

¿Cómo funciona la poda? El eslabón más débil

- **Poda del Eslabón Más Débil (Weakest Link Pruning):** Para decidir qué rama cortar, el algoritmo no prueba al azar, sino que evalúa una tasa de efectividad para cada nodo.
- Se calcula el balance entre la reducción de error y el costo en hojas extra:

$$\frac{RSS(nodo) - RSS(subarbol)}{|T| - 1}$$

Numerador: Cuánto error nos ahorra esa rama.

Denominador: Cuántas hojas extra nos cuesta

- **La Selección:** El nodo con el valor más bajo es el eslabón más débil. Al aumentar α , esa es la primera rama en caer.
- **Consecuencia (anidado y predecible):** Al recortar siempre la peor rama del árbol actual, nunca se generan ramas nuevas. Esto crea una secuencia finita, ordenada y contenida de subárboles candidatos:

$$T_0 \supset T_1 \supset T_2 \supset \dots \supset T_{raiz}$$

ÁRBOLES DE REGRESIÓN

Algoritmo completo

1. Usando *Recursive binary splitting* se crea el árbol más grande con el dataset de entrenamiento. El entrenamiento termina cuando cada hoja tenga menos de un número determinado de observaciones.
2. Se aplica la técnica de podado de árbol para obtener un set de subárboles como función de α . Para elegir α se usa validación cruzada.
3. Se elige el subárbol del paso (2) que corresponde al valor de α que disminuya el error.



Vamos a practicar un
poco ...

ÁRBOL DE CLASIFICACIÓN

ÁRBOLES DE CLASIFICACIÓN

Un árbol de clasificación es muy similar a uno de regresión, pero ahora se usa para predecir una **variable cualitativa**.

En el caso de regresión, al llegar la hoja, obteníamos el valor con el promedio de los valores en la hoja. Ahora, obtenemos la clase en base a la **clase que más ocurre** en las muestras que están en la hoja.

Al interpretar los resultados de un **árbol de clasificación**, a menudo estamos interesados no sólo en la predicción de clase correspondiente a una región de nodo terminal particular, sino también en las **proporciones de clase entre las observaciones de entrenamiento** que caen en esa región.

ÁRBOLES DE CLASIFICACIÓN

La forma más simple de crear un **árbol de clasificación** es muy parecida a la del árbol de regresión, con la estrategia **top-down** y **greedy**, aunque ahora no contamos con el error cuadrático.

Una primera métrica que podemos usar es el **error de clasificación**:

$$E = 1 - \max_k (p_{mk})$$

donde $\hat{p}_{mk}$ es la proporción de las observaciones de entrenamiento en la región m que son de la clase k .

Esta métrica mide la fracción de observaciones que no pertenecen a la clase mayoritaria de esa región.

ÁRBOLES DE CLASIFICACIÓN

El **error de clasificación** no es lo suficientemente sensible para decidir las divisiones del árbol.

Ejemplo: en una hoja del árbol con 800 observaciones: 400 son clase A y 400 clase B. Como hay un empate (50/50), si se usa el árbol así el error de clasificación es 0.50.

Ahora el algoritmo prueba dos divisiones distintas para ver cuál es mejor:

División 1:

- Hoja Izquierda: 300 de A y 100 de B. (Predice A. Error = 100)
- Hoja Derecha: 100 de A y 300 de B. (Predice B. Error = 100)
- Error total de esta división: 200 errores sobre 800 = 0.25 (25%).

División 2:

- Hoja Izquierda: 200 de A y 400 de B. (Predice B. Error = 200)
- Hoja Derecha: 200 de A y 0 de B. (Predice A. Error = 0)
- Error total de esta división: 200 errores sobre 800 = 0.25 (25%).

ÁRBOLES DE CLASIFICACIÓN

El **error de clasificación** no es lo suficientemente sensible para decidir las divisiones del árbol.

Ejemplo: en una hoja del árbol con 800 observaciones: 400 son clase A y 400 clase B. Como hay un empate (50/50), si se usa el árbol así el error de clasificación es 0.50.

Ahora el algoritmo prueba dos divisiones distintas para ver cuál es mejor:

División 1:

- Hoja Izquierda: 300 de A y 100 de B. (Predice A. Error = 100)
- Hoja Derecha: 100 de A y 300 de B. (Predice B. Error = 100)
- Error total de esta división: 200 errores sobre 800 = 0.25 (25%).

División 2:

- Hoja Izquierda: 200 de A y 400 de B. (Predice B. Error = 200)
- Hoja Derecha: 200 de A y 0 de B. (Predice A. Error = 0)
- Error total de esta división: 200 errores sobre 800 = 0.25 (25%).

Vemos que este caso es mejor
pero esta métrica no puede
verlo

ÁRBOLES DE CLASIFICACIÓN

Por eso, en la práctica se utilizan otras medidas.

Vamos a ver dos **medidas de impureza**:

- el índice de Gini
- la entropía

ÁRBOLES DE CLASIFICACIÓN

El **índice de Gini** mide cuán mezcladas están las clases dentro de un nodo. Es una medida de la impureza de un nodo:

$$G = \sum_{k=1}^K \hat{p}_{mk} (1 - \hat{p}_{mk})$$

Cuando las $\hat{p}_{mk}$ son cercanas a 0 o 1, significa que en el nodo predomina una o pocas clases. En este caso, el índice toma valores bajos y el nodo se considera más puro.

El índice de Gini se usó inicialmente para medir la desigualdad de los países.

ÁRBOLES DE CLASIFICACIÓN

En un árbol de clasificación queremos encontrar una división que aporte la mayor cantidad de información posible.

Si una regla hace que las observaciones de una clase vayan hacia un lado y las de otra clase hacia el otro, será una muy buena división.

En cambio, si la regla apenas modifica la distribución de las clases, probablemente no sea una buena elección.

Esta noción de **cantidad de información** se mide mediante la **entropía**, que también puede interpretarse como una medida del desorden.

ÁRBOLES DE CLASIFICACIÓN

La definición de **entropía** es:

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log(\hat{p}_{mk})$$

Cuando las $\hat{p}_{mk}$ son similares entre sí, la entropía es alta y existe mayor incertidumbre sobre la clase de una observación. Si, por el contrario, todas las observaciones de entrenamiento pertenecen a una sola clase, la entropía es cero.

ÁRBOLES DE CLASIFICACIÓN

Volvamos al ejemplo pero usando el índice de Gini.

Ejemplo: en una hoja del árbol con 800 observaciones: 400 son clase A y 400 clase B.

División 1:

- Hoja Izquierda: Prop. 0.75 de A y 0.25 de B. **$G_{izq} = 0.375$**
- Hoja Derecha: Prop. 0.25 de A y 0.75 de B. **$G_{der} = 0.375$**
- Como ambas hojas tienen 400 observaciones (la mitad exacta cada una), el promedio ponderado es simple: **$G = 0.375$**

División 2:

- Hoja Izquierda: Prop. 0.33 de A y 0.66 de B. **$G_{izq} = 0.444$**
- Hoja Derecha: Prop. 1.0 de A y 0.0 de B. **$G_{der} = 0$**
- Acá multiplicamos el Gini de cada hoja por su peso (600/800 para la izquierda, 200/800 para la derecha): **$G = 0.333$**

ÁRBOLES DE CLASIFICACIÓN

Volvamos al ejemplo pero usando el índice de Gini.

Ejemplo: en una hoja del árbol con 800 observaciones: 400 son clase A y 400 clase B.

División 1:

- Hoja Izquierda: Prop. 0.75 de A y 0.25 de B. **G_{izq} = 0.375**
- Hoja Derecha: Prop. 0.25 de A y 0.75 de B. **G_{der} = 0.375**
- Como ambas hojas tienen 400 observaciones (la mitad exacta cada una), el promedio ponderado es simple: **G = 0.375**

División 2:

- Hoja Izquierda: Prop. 0.33 de A y 0.66 de B. **G_{izq} = 0.444**
- Hoja Derecha: Prop. 1.0 de A y 0.0 de B. **G_{der} = 0**
- Acá multiplicamos el Gini de cada hoja por su peso (600/800 para la izquierda, 200/800 para la derecha): **G = 0.333**

ÁRBOLES DE CLASIFICACIÓN

Ventajas y desventajas



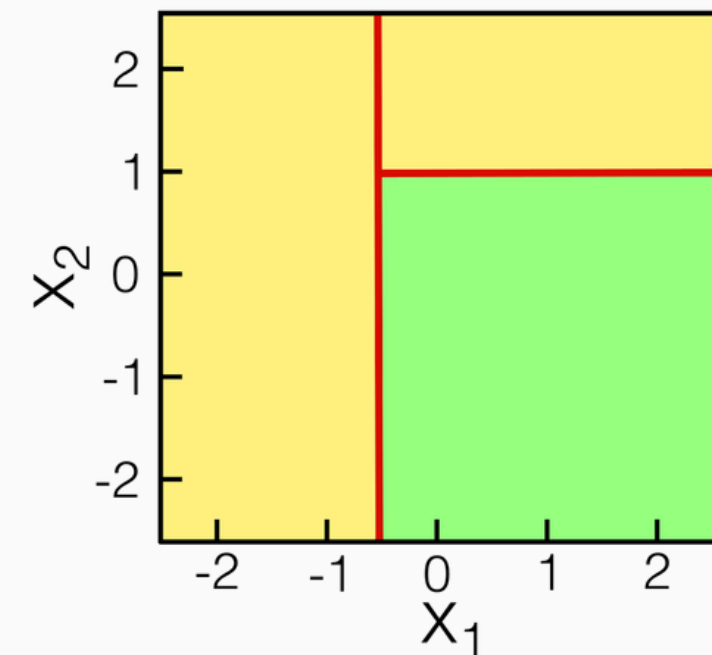
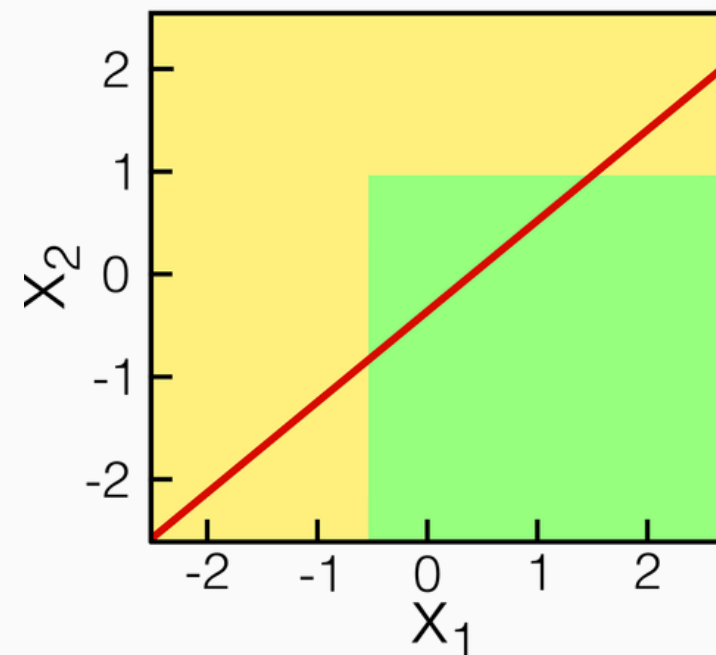
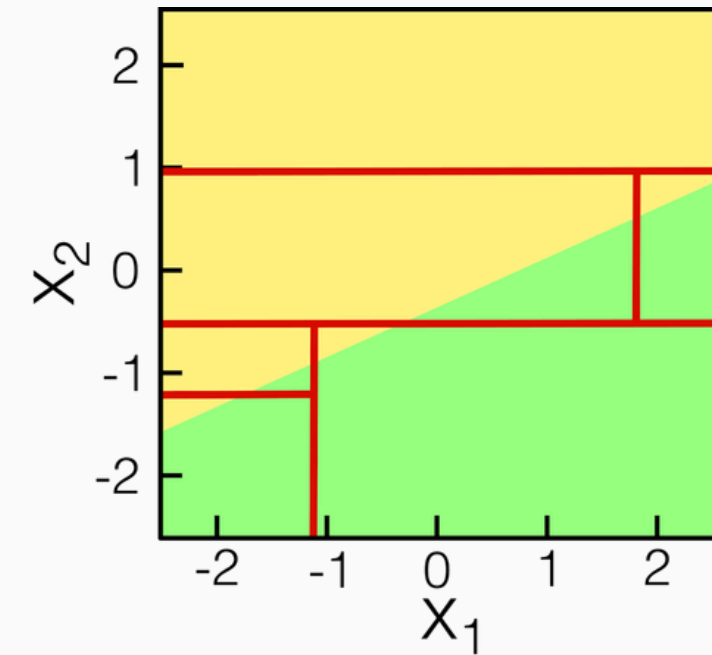
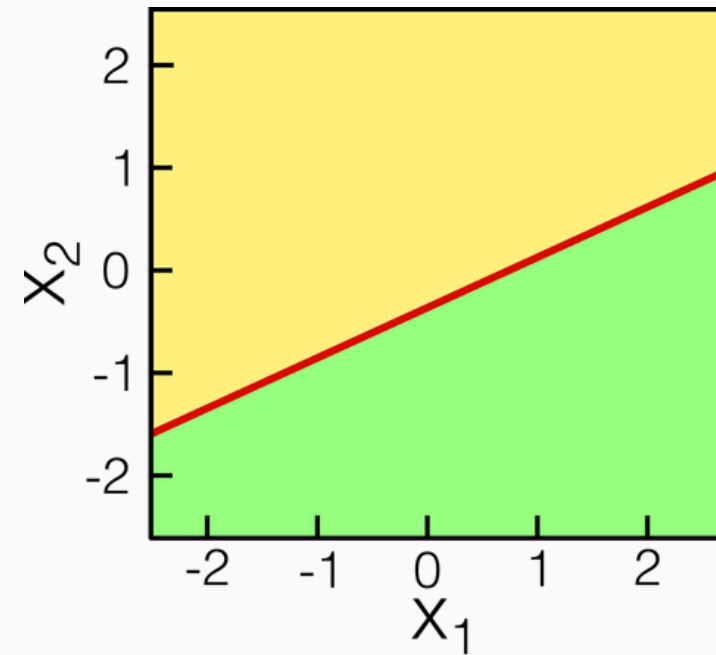
- Son fáciles de explicar a las personas, más inclusive que la regresión lineal.
- Hipotéticamente, se acercan más a la forma que un humano piensa.
- Se pueden representar gráficamente.
- Pueden manejar fácilmente variables cualitativas sin necesidad de crear variables dummy.



- No tienen el mismo nivel de exactitud de predicción que otros modelos.
- No son robustos; pequeños cambios en los datos pueden cambiar grandes cambios en la predicción.

ÁRBOLES DE CLASIFICACIÓN

Ventajas y desventajas





Vamos a practicar un
poco ...

En la próxima clase veremos ...

Métodos de ensamble. Modelos que votan. Bagging. Bosques aleatorios. Boosting.